# TypeScript

## The Problem in JavaScript

JavaScript is a dynamically typed language.

This means:

- Variables can hold any type.
- Functions accept any type.
- Errors are discovered only when the program runs.

Let’s look at an example:

```javascript
function add(a, b) {
  return a + b;
}

add(5, 3);      // 8
add("5", 3);    // "53"
```

JavaScript does not complain.

But in real applications, this flexibility becomes dangerous.

Another example:

```javascript
function greet(user) {
  console.log("Hello " + user.name);
}

greet({});
```

This prints:

```none
Hello undefined
```

The program does not crash immediately, but it behaves incorrectly.

In large systems, these small mistakes can create serious bugs that are hard to trace.

### The Core Issue

JavaScript:

- Does not verify function argument types.
- Does not enforce object structure.
- Does not validate return types.

It trusts the developer completely.

In small projects this may be fine.
In large teams and enterprise applications, this becomes risky.

We need something that helps us catch mistakes earlier.

# What is TypeScript?

TypeScript is a superset of JavaScript.

That means:

- Everything valid in JavaScript is valid in TypeScript.
- TypeScript adds static typing and additional tooling.

Static typing means:

Types are checked during development, before the program runs.

Example:

```typescript
function add(a: number, b: number) {
  return a + b;
}

add("5", 3); // ❌ Error
```

Now the mistake is detected immediately in the editor.

TypeScript does not replace JavaScript.
It improves the development experience.

# How TypeScript Works

Browsers and Node.js do not understand TypeScript directly.

TypeScript must be compiled into JavaScript.

The workflow is:

TypeScript file (.ts)
→ TypeScript compiler (tsc)
→ JavaScript file (.js)
→ Executed by browser or Node.js

Example:

TypeScript:

```typescript
let age: number = 25;
```

After compilation:

```javascript
let age = 25;
```

Notice something important:

Types disappear.

They are used only during development.

This means:

- No runtime performance cost.
- No extra code in production.

# The Conflict Problem After Building

When we compile a TypeScript file like this:

```shell
tsc start.ts
```

TypeScript generates:

```none
start.ts
start.js
```

Now both files exist in the same folder.

Sometimes we may see an error like:

Duplicate function implementation.

Why does this happen?

# Why This Conflict Happens

By default, if a TypeScript file does not contain `import` or `export`,
TypeScript treats it as a global script.

Global scripts share the same global scope.

If:

- `start.ts` contains a function named `add`
- `start.js` (the compiled output) also contains `add`
- Both are being analyzed by TypeScript

Then TypeScript thinks:

"There are two functions with the same name in the global scope."

So it reports:

Duplicate function implementation.

Even though logically we only wrote it once.

This happens because:

TypeScript sees both files as part of the same project and merges their global scope.

# How to Solve the Conflict (Step by Step)

There are two main solutions.

## Solution 1 — Convert the File into a Module (Recommended)

Add this line at the bottom of your TypeScript file:

```typescript
export {};
```

Now your file becomes a module.

Modules have their own private scope.

This isolates the file and prevents global collisions.

### Why This Works

When a file contains `import` or `export`, TypeScript treats it as a module.

Modules:

- Do not share global scope.
- Do not conflict with other files.
- Are the standard structure in real projects.

## Solution 2 — Use tsconfig.json with an Output Folder (Best Practice)

Instead of generating `.js` files in the same folder, we configure TypeScript properly.

Create a configuration file:

```shell
tsc --init
```

Then modify `tsconfig.json`:

```json
{
  "compilerOptions": {
    "outDir": "./dist",
    "rootDir": "./src"
  }
}
```

Project structure:

```none
src/
   start.ts
dist/
   start.js
```

Now:

- TypeScript source files live in `src`
- Compiled JavaScript goes into `dist`
- No more conflicts

This is the professional setup.

# Why This Is Important

This conflict teaches us something important about TypeScript:

- Files without `import` or `export` are global scripts.
- Global scope can cause collisions.
- Real projects use modules.
- Proper project structure prevents errors.

Understanding this early builds strong architectural habits.

# TypeScript Build Setup: `rootDir`, `outDir`, Running, and `tsc --watch`

## Why do we even need `rootDir` and `outDir`?

When you run TypeScript, you have **two worlds**:

- **Source world**: your TypeScript code (`.ts`) — for developers
- **Output world**: compiled JavaScript (`.js`) — for Node/browser

If you compile `.ts` files inside the same folder, you may end up with:

- `start.ts`
- `start.js`

side by side → and sometimes this causes **conflicts**, confusion, and messy projects.

So we organize the project like professionals:

Put TypeScript in a **source folder**
Put compiled JavaScript in a **build folder**

That is exactly what `rootDir` and `outDir` help us do.

# What is `rootDir`?

## Meaning

`rootDir` is:
**Where your TypeScript source files live**

Example:

```JSON
"rootDir": "./src"
```

That means:

- TypeScript expects your `.ts` files to be inside `src/`
- This becomes the “base folder” for your project structure

### Why does it matter?

Because TypeScript needs to understand:

- which folder is the “original code”
- and how to preserve the same folder structure when building

# What is `outDir`?

## Meaning

`outDir` is:
**Where TypeScript will put the compiled JavaScript files**

Example:

```JSON
"outDir": "./dist"
```

That means:

- All compiled `.js` files go into `dist/`
- Your source stays clean inside `src/`

## The clean project structure (recommended)

```None
project-folder/
  src/
    start.ts
  dist/
    start.js   <-- auto generated
  tsconfig.json
  package.json
```

`src` is what you write
`dist` is what you run

## How to compile now (and what happens behind the scenes)

### When you run:

```Shell
tsc
```

(TypeScript does NOT need a file name if you have `tsconfig.json`)

### Behind the scenes, TypeScript does these steps:

1. **Reads `tsconfig.json`**
- it loads compiler settings like `rootDir`, `outDir`, etc.
2. **Finds all `.ts` files**
- usually inside `rootDir` (like `src/`)
3. **Type-checks your code**
- checks: types, function arguments, object shapes, etc.
- if errors exist → it reports them
4. **If no blocking errors**
- it compiles `.ts` → `.js`
5. **Writes compiled files into `outDir`**
- it keeps folder structure the same

Example:

- `src/start.ts` → `dist/start.js`

That's why `tsc` is both:

- a **type-checker**
- and a **compiler**

## How do we run our program after compiling?

After `tsc` creates JS in `dist/`, we run with Node:

```shell
node dist/start.js
```

Node runs **JavaScript**, not TypeScript.

So we always run the compiled output.

## The problem: "I hate compiling every time"

When you edit `start.ts`, you must compile again:

- change code
- `tsc`
- `node dist/start.js`

That becomes annoying during development.

# The solution: `tsc --watch`

## What it means

`--watch` tells TypeScript:

"Keep watching my `.ts` files, and whenever I change something, recompile automatically."

Run:

```shell
tsc --watch
```

Now TypeScript stays running and prints messages like:

- "Watching for file changes…"
- "File change detected. Starting incremental compilation…"

## What happens behind the scenes with watch?

1. TypeScript starts a **watch process**
2. It monitors files inside `rootDir`
3. When a file changes:
    - it re-checks types
    - recompiles only what changed (fast!)
4. It updates `dist/` automatically

So you don't need to manually run `tsc` every time.

# How to use watch properly (your daily workflow)

## Terminal 1 (watch mode)

```shell
tsc --watch
```

Leave it running.

## Terminal 2 (run Node)

Whenever you want to execute:

```shell
node dist/start.js
```

So the workflow becomes:

- you edit TypeScript
- TypeScript compiles automatically
- you just run Node when you want to test

## Even better: add scripts in `package.json`

In `package.json`, add:

```json
{
  "scripts": {
    "build": "tsc",
    "watch": "tsc --watch",
    "start": "node dist/start.js"
  }
}
```

Now you can run:

```shell
npm run watch
```

and in another terminal:

```shell
npm run start
```

This is clean and professional.

# TypeScript Basic Types

In this section we will learn:

- `string`
- `number`
- `boolean`
- `array`
- arrays with multiple types
- `any`
- when and why to use `any`

# What Is a Type?

A **type** describes the kind of value a variable can hold.

In JavaScript:

```javascript
let age = 25;
age = "hello";
```

This is allowed.

In TypeScript, we can define what kind of value a variable must hold.

This helps us:

- prevent mistakes
- make code predictable
- document our intentions

# String Type

A `string` represents text values.

Example:

```
TypeScript
let name: string = "Dania";
```

Now `name` must always be text.

If we try:

```
TypeScript
name = 5; // ❌ Error
```

TypeScript prevents invalid assignment.

### Why is this useful?

Because it guarantees that:

- `name` will always behave like text
- string methods are safe to use

Example:

```
TypeScript
console.log(name.toUpperCase());
```

TypeScript knows `name` is a string, so it allows string methods.

# Number Type

A `number` represents numeric values (integers and decimals).

Example:

```typescript
let age: number = 25;
let price: number = 99.99;
```

Invalid example:

```typescript
age = "twenty five"; // ❌ Error
```

### Important

In TypeScript, there is only one numeric type: `number`.

It includes:

- integers
- floats
- negative numbers

# Boolean Type

A `boolean` represents true/false values.

Example:

```typescript
let isLoggedIn: boolean = true;
let hasPermission: boolean = false;
```

Invalid example:

```typescript
isLoggedIn = "yes"; // ❌ Error
```

### Why booleans are important

Booleans control:

- conditions

- permissions
- states
- logic flow

# Arrays in TypeScript

An array is a collection of values.

## Method 1: Using `type[]`

```TypeScript
let numbers: number[] = [1, 2, 3, 4];
```

This means:

- Every element must be a number.

Invalid example:

```TypeScript
numbers.push("hello"); // ❌ Error
```

## Method 2: Using Generic Syntax

```TypeScript
let names: Array<string> = ["Ali", "Sara", "Lina"];
```

This means the same thing:

- It is an array of strings.

Both syntaxes are valid.

# Arrays with Multiple Types

Sometimes we need arrays that contain different types.

Example:

```
TypeScript
let mixed: (string | number)[] = ["Ali", 25, "Sara", 30];
```

Here we use:

`string | number`

This is called a **union type**.

It means:

- Each element can be either string OR number.

Invalid example:

```
TypeScript
mixed.push(true); // ❌ Error
```

Because `boolean` was not included in the union.

# What is any?

The `any` type disables type checking.

Example:

```
TypeScript
let data: any = 5;

data = "hello";
data = true;
data = { name: "Ali" };
```

Everything is allowed.

TypeScript stops protecting that variable.

# When Should We Use any?

any should be used carefully.

### Good cases for any:

1. When working with external libraries that do not have types.
2. When gradually migrating JavaScript to TypeScript.
3. When temporarily bypassing strict typing during debugging.
4. When handling dynamic data where structure is truly unknown.

Example:

```
TypeScript
function logData(data: any) {
  console.log(data);
}
```

# Why Avoid Using any Too Much?

If you use any everywhere:

```
TypeScript
let value: any = "hello";
value.toFixed(2); // No error, but may crash at runtime
```

You lose:

- type safety
- autocomplete intelligence
- error detection

Using any makes TypeScript behave like JavaScript.

So it should be used only when necessary.

# Function Types in TypeScript

Functions are one of the most important parts of any program.

In JavaScript, functions are flexible — sometimes too flexible.
TypeScript allows us to define **exactly what a function expects and what it returns**.

This creates safety, clarity, and predictability.

# Why Do We Need Function Types?

In JavaScript:

```javascript
function add(a, b) {
  return a + b;
}

add("5", 3);
```

JavaScript does not complain.

But what if we wanted `add` to only work with numbers?

There is no built-in protection.

In TypeScript, we define a **contract** for the function.

# Typing Function Parameters

Basic syntax:

```typescript
function add(a: number, b: number) {
  return a + b;
}
```

Explanation:

- `a: number` → parameter `a` must be a number
- `b: number` → parameter `b` must be a number

Now:

```
TypeScript
add(5, 3);       // ✅ Valid
add("5", 3);     // ❌ Error
```

TypeScript checks argument types **before execution**.

# Typing Return Values

We can also define what the function returns.

```
TypeScript
function add(a: number, b: number): number {
  return a + b;
}
```

The `: number` after parentheses means:

This function must return a number.

### What happens if we return the wrong type?

```
TypeScript
function add(a: number, b: number): number {
  return "hello"; // ❌ Error
}
```

TypeScript protects both:

- the input
- the output

This is powerful because it guarantees consistent behavior.

# Return Type Inference

TypeScript is smart.

If we don't write the return type:

```
TypeScript
function multiply(a: number, b: number) {
  return a * b;
}
```

TypeScript automatically infers the return type as `number`.

You can hover over the function to see the inferred type.

### When should we explicitly define return types?

- For exported functions
- For complex logic
- When building APIs
- For readability

Explicit return types make code more predictable.

# Void Functions

Some functions don't return a value.

Example:

```
TypeScript
function logMessage(message: string): void {
  console.log(message);
}
```

`void` means:

The function does not return anything.

If we try:

```typescript
function logMessage(message: string): void {
  return 5; // ❌ Error
}
```

TypeScript prevents it.

# Optional Parameters

Sometimes a parameter is not required.

```typescript
function greet(name: string, age?: number) {
  console.log(name, age);
}
```

The `?` means:

`age` may or may not be provided.

Usage:

```typescript
greet("Dania");        // ✅
greet("Dania", 25);    // ✅
```

Important rule:

Optional parameters must come after required ones.

# Default Parameters

Instead of optional parameters, we can assign a default value.

```TypeScript
function greet(name: string, age: number = 20) {
  console.log(name, age);
}
```

If `age` is not provided, it defaults to `20`.

Difference between optional and default:

- Optional → value may be `undefined`
- Default → value always has something

# Arrow Functions with Types

Arrow functions follow the same typing rules.

```TypeScript
const subtract = (a: number, b: number): number => {
  return a - b;
};
```

Everything works the same:

- typed parameters
- typed return

# Function Type as a Variable

Functions themselves have types.

Example:

```typescript
let calculate: (a: number, b: number) => number;
```

Explanation:

This means:

`calculate` is a function
It accepts two numbers
It returns a number

Now we can assign a function:

```typescript
calculate = function(a, b) {
  return a + b;
};
```

If we try:

```typescript
calculate = function(a, b) {
  return "hello";
}; // ❌ Error
```

TypeScript enforces the function shape.

# Functions as Parameters (Callbacks)

Sometimes functions accept other functions.

Example:

```
TypeScript
function processNumbers(
  a: number,
  b: number,
  operation: (x: number, y: number) => number
): number {
  return operation(a, b);
}
```

Usage:

```
TypeScript
processNumbers(5, 3, (x, y) => x + y); // ✅
processNumbers(5, 3, (x, y) => "hello"); // ❌ Error
```

This ensures callback correctness.

# Real-World Example

Let’s define a user creation function:

```
TypeScript
function createUser(name: string, age: number): { name:
string; age: number } {
  return { name, age };
}
```

Now TypeScript guarantees:

- Name must be string
- Age must be number
- Returned object must match structure

If we forget a property:

```
TypeScript
return { name }; // ❌ Error
```

TypeScript catches it.

# Interfaces in TypeScript

# Real Scenario: Why Do We Need Interfaces?

Imagine we are building a system for managing users.

We write this function:

```
TypeScript
function printUser(user: { name: string; age: number; isAdmin:
boolean }) {
  console.log(`${user.name} is ${user.age} years old.`);
}
```

Now we use it:

```
TypeScript
printUser({
  name: "Dania",
  age: 28,
  isAdmin: true
});
```

Everything works.

### But now imagine…

We need the same user structure in multiple places:

- In `createUser`
- In `updateUser`
- In `deleteUser`
- In `loginUser`
- In `getUserProfile`

So we repeat the same type over and over:

```typescript
function createUser(user: { name: string; age: number;
isAdmin: boolean }) {}

function updateUser(user: { name: string; age: number;
isAdmin: boolean }) {}

function deleteUser(user: { name: string; age: number;
isAdmin: boolean }) {}
```

Now we have a problem.

# The Problem Without Interfaces

If tomorrow we decide:

We want to add a new property:

```typescript
email: string
```

Now we must update every function manually.

If we forget one?

Bug.

If we mistype a property name in one place?

Bug.

If someone writes:

```
TypeScript
isadmin
```

instead of

```
TypeScript
isAdmin
```

Bug.

This is where we realize:

We need a **single source of truth** for this object shape.

That is exactly what an **interface** gives us.

# What Is an Interface?

An **interface** is a blueprint that defines the structure of an object.

It describes:

- What properties exist
- What types they have
- What is required
- What is optional

It does NOT create values.

It only describes structure.

# Creating Our First Interface

Let’s define our user structure properly.

```
TypeScript
interface User {
  name: string;
  age: number;
  isAdmin: boolean;
```

```
}
```

Now we can rewrite our functions cleanly:

```TypeScript
function createUser(user: User) {}

function updateUser(user: User) {}

function deleteUser(user: User) {}
```

Much cleaner.

# Why This Is Powerful

Now, if we change the interface:

```TypeScript
interface User {
  name: string;
  age: number;
  isAdmin: boolean;
  email: string;
}
```

Everywhere that uses `User` will automatically update.

If something is missing:

TypeScript shows an error.

This is maintainability.

# Example: Missing Property Error

```typescript
const user: User = {
  name: "Dania",
  age: 28,
  isAdmin: true
};
```

TypeScript error:

Property 'email' is missing.

This prevents inconsistent data.

# Optional Properties

Sometimes not all properties are required.

Example:

```typescript
interface User {
  name: string;
  age: number;
  isAdmin: boolean;
  email?: string;
}
```

The `?` makes the property optional.

Now this works:

```typescript
const user: User = {
  name: "Dania",
  age: 28,
  isAdmin: true
};
```

Optional means:

It may exist
It may not exist

# Readonly Properties

Sometimes we don't want properties to change.

Example:

```typescript
interface User {
  readonly id: number;
  name: string;
  age: number;
}
```

Now:

```typescript
const user: User = {
  id: 1,
  name: "Ali",
  age: 25
};

user.id = 2; // ❌ Error
```

`readonly` protects critical data.

# Interface for Functions

Interfaces can also describe function shapes.

Example:

```TypeScript
interface MathOperation {
  (a: number, b: number): number;
}
```

Usage:

```TypeScript
const add: MathOperation = (a, b) => a + b;
```

This ensures consistency.

# Real-World Example: API Response

Imagine backend returns:

```TypeScript
{
  id: 1,
  title: "Learn TypeScript",
  completed: false
}
```

Define interface:

```TypeScript
interface Todo {
  id: number;
  title: string;
  completed: boolean;
}
```

Now:

```typescript
function printTodo(todo: Todo) {
  console.log(todo.title);
}
```

TypeScript guarantees structure consistency.

# Why Interfaces Matter in Real Projects

Interfaces:

- Prevent data inconsistency
- Improve team collaboration
- Create reusable type definitions
- Improve autocomplete
- Reduce duplication
- Make refactoring safe

They are essential in:

- React props
- API data
- Backend models
- Service layers
- Microservices

# Type Alias vs Interface

# Real Scenario – When Do We Need a Type Alias?

Imagine we are building a payment system.

Each order has a status.

Possible statuses are:

- "pending"
- "paid"
- "cancelled"

- "refunded"

In JavaScript, we might write:

```
TypeScript
let status = "pending";
```

But this is dangerous.

What if someone writes:

```
TypeScript
status = "pendng"; // typo
status = "done";   // invalid value
```

JavaScript will not complain.

We want to restrict this variable to specific allowed values.

We can create a type alias:

```
TypeScript
type OrderStatus = "pending" | "paid" | "cancelled" |
"refunded";
```

Now:

```
TypeScript
let status: OrderStatus = "pending"; // ✅
status = "done"; // ❌ Error
```

TypeScript now protects us.

## Why is this powerful?

Because:

- We defined a reusable name.
- We restricted allowed values.
- We improved readability.
- We removed magic strings.

This is a perfect example of when we **need** a type alias.

# What Is a Type Alias?

A type alias is:

A custom name for a type.

It allows us to:

- Give a name to primitive types
- Give a name to object structures
- Create union types
- Create function types
- Simplify complex types

Syntax:

```typescript
type Name = TypeDefinition;
```

Example with object:

```typescript
type User = {
  id: number;
  name: string;
  isAdmin: boolean;
};
```

Now:

```typescript
function printUser(user: User) {
  console.log(user.name);
}
```

Instead of rewriting the object structure everywhere.

# When Should We Use a Type Alias?

Type alias is especially useful when:

- You need union types
- You need function types
- You want to combine types
- You want to simplify complex types
- You want semantic naming for primitives

Example:

```TypeScript
type UserID = number;
type Role = "admin" | "user" | "guest";
```

Even though `UserID` is just a number, it gives meaning.

# What Is an Interface?

An interface is:

A blueprint for an object structure.

It describes the shape of an object.

Example:

```TypeScript
interface User {
  id: number;
  name: string;
  isAdmin: boolean;
}
```

Usage:

```TypeScript
function printUser(user: User) {
  console.log(user.name);
```

```
}
```

So far, it looks very similar to `type`.

# So What's the Difference?

Let's break it clearly.

## 1. Object Shapes

Both can define object structures:

```TypeScript
type User = {
  name: string;
  age: number;
};
```

```TypeScript
interface User {
  name: string;
  age: number;
}
```

For basic object shapes → both work the same.

## 2. Union Types

Type alias supports unions:

```TypeScript
type Status = "pending" | "done";
```

Interface cannot do this:

```typescript
interface Status = "pending" | "done"; // ❌ Not allowed
```

This is a major difference.

## 3. Function Types

Type alias supports function types directly:

```typescript
type MathOperation = (a: number, b: number) => number;
```

Interface can also define function shapes, but syntax is slightly different:

```typescript
interface MathOperation {
  (a: number, b: number): number;
}
```

So both can handle functions, but type alias is often cleaner.

## 4. Extending / Inheritance

Both support extension:

### Interface:

```typescript
interface Person {
  name: string;
}

interface Employee extends Person {
  salary: number;
}
```

### Type:

```typescript
type Person = {
  name: string;
};

type Employee = Person & {
  salary: number;
};
```

Notice:

- Interface uses `extends`
- Type uses `&` (intersection)

## 5. Declaration Merging (Important Difference)

Interfaces support merging.

Example:

```typescript
interface User {
  name: string;
}

interface User {
  age: number;
}
```

TypeScript merges them into:

```typescript
interface User {
  name: string;
  age: number;
}
```

Type alias does NOT allow this:

```typescript
type User = { name: string };
type User = { age: number }; // ❌ Error
```

This makes interface more flexible for large-scale architecture.

# So When Do We Use Each One?

### Use `interface` when:

- Defining object models
- Building APIs
- Defining class contracts
- Designing application architecture

### Use `type` when:

- Creating union types
- Creating intersection types
- Creating function types
- Simplifying complex types
- Giving semantic names to primitives

# Enums in TypeScript — Detailed Explanation

## Why Do Enums Exist?

Imagine we are building a delivery system.

An order can have these states:

- Created
- Shipped
- Delivered
- Cancelled

In JavaScript, we might write:

```typescript
let status = "created";
```

Later:

```typescript
if (status === "delivered") {
  console.log("Order complete");
}
```

### The Problems

1. Typos are not detected:

```typescript

status = "delivred";
```

2. Anyone can assign invalid values:

```typescript
status = "unknown";
```

3. There is no centralized definition of allowed values.

We need:

- A fixed set of allowed values
- Readable names
- Centralized control

That's exactly what **enum** provides.

# What Is an Enum?

Enum stands for **enumeration**.

It is a way to define a group of related constant values.

It allows us to:

- Define named constants
- Restrict a variable to specific values
- Improve readability
- Prevent invalid assignments

# Numeric Enums (Default Behavior)

Basic enum:

```typescript
enum OrderStatus {
  Created,
  Shipped,
  Delivered,
  Cancelled
}
```

By default, TypeScript assigns numeric values starting from 0.

This becomes:

```
TypeScript
Created = 0
Shipped = 1
Delivered = 2
Cancelled = 3
```

## Usage:

```
TypeScript
let status: OrderStatus = OrderStatus.Created;
```

# Reverse Mapping (Numeric Enums Only)

Numeric enums support reverse lookup.

Example:

```
TypeScript
console.log(OrderStatus.Created); // 0
console.log(OrderStatus[0]);      // "Created"
```

Why?

Because TypeScript generates mapping in both directions.

This only works for numeric enums.

# Custom Numeric Values

We can manually assign numbers:

```
TypeScript
enum OrderStatus {
  Created = 10,
  Shipped = 20,
```

```
  Delivered = 30
}
```

Now values are:

- Created → 10
- Shipped → 20
- Delivered → 30

# String Enums (More Practical)

Instead of numbers, we can use strings:

```TypeScript
enum OrderStatus {
  Created = "created",
  Shipped = "shipped",
  Delivered = "delivered"
}
```

Usage:

```TypeScript
let status: OrderStatus = OrderStatus.Created;
```

Now the value is:

```TypeScript
"created"
```

## Why String Enums Are Often Better

- More readable in logs
- Safer for APIs
- No accidental numeric confusion
- No reverse mapping complexity

In real-world projects, string enums are often preferred.

# Enums in Functions

Example:

```typescript
function updateOrder(status: OrderStatus) {
  console.log("Status:", status);
}

updateOrder(OrderStatus.Delivered); // ✅
updateOrder("delivered"); // ❌ Error
```

This enforces strict control.

# Const Enums

We can define:

```typescript
const enum OrderStatus {
  Created,
```

```
  Shipped,
  Delivered
}
```

Difference:

- No JavaScript object is generated.
- Values are inlined at compile time.

Example:

```typescript
let status = OrderStatus.Created;
```

Becomes in JS:

```javascript
let status = 0;
```

This improves performance slightly.

But:

- Cannot be used in some runtime scenarios.
- Often avoided in libraries.

# Enum vs Union Types

Modern TypeScript often prefers union types instead of enums.

Example using union:

```typescript
type OrderStatus = "created" | "shipped" | "delivered";
```

This does NOT generate runtime code.

It is purely type-level.

# Generics in TypeScript

## 1. Introduction: The Problem We Want to Solve

When writing programs, we often create functions or classes that work with **different types of data**.

For example, imagine we want a function that simply returns the value we give it.

### JavaScript Example

```javascript
function identity(value) {
  return value;
}

console.log(identity(5));      // 5
console.log(identity("hello")) // "hello"
```

This works, but JavaScript does **not enforce any type rules**.

The function accepts **anything**, and it also returns **anything**.

When projects become large, this flexibility becomes risky.

For example:

```javascript
function identity(value) {
  return value;
}

let result = identity("hello");
result.toFixed(2); // Runtime error
```

The program crashes because `"hello"` is a string and does not have `toFixed()`.

JavaScript does not warn us beforehand.

## 2. Trying to Solve It With any

In TypeScript we might try this:

```typescript
function identity(value: any): any {
  return value;
}
```

The problem:

- any disables type checking.
- TypeScript stops protecting us.

Example:

```typescript
let result = identity("hello");
result.toFixed(2); // ❌ No TypeScript error, but runtime crash
```

Using any removes the benefit of TypeScript.

So we need something better.

# 3. The Idea of Generics

Generics allow us to write **reusable code** that works with many types while **preserving type safety**.

Instead of fixing the type beforehand, we allow the **type to be decided when the function is used**.

Think of generics like a **placeholder for a type**.

The placeholder is usually written as:

```
None
<T>
```

Where `T` means **Type**.

# 4. Our First Generic Function

```typescript
function identity<T>(value: T): T {
  return value;
}
```

Explanation:

| Part | Meaning |
|---|---|
| `<T>` | This function is generic |
| `value: T` | The input has type T |
| `: T` | The function returns the same type |

This means:

> Whatever type you pass in, the function returns the same type.

## Example Usage

```typescript
const numberResult = identity<number>(5);
const stringResult = identity<string>("hello");
```

TypeScript understands that:

- `numberResult` is a number
- `stringResult` is a string

# 5. Type Inference

Usually we don't even need to specify the type manually.

TypeScript can **infer the type automatically**.

```typescript
const result1 = identity(10);      // TypeScript infers number
const result2 = identity("hello"); // TypeScript infers string
```

TypeScript looks at the value we pass and determines the type.

# 6. Generics with Arrays

Generics are very useful when working with arrays.

Suppose we want a function that returns the **first element of an array**.

### Without Generics

```typescript
function firstNumber(arr: number[]): number {
  return arr[0];
}
```

But this only works for numbers.

We would need separate functions for strings, booleans, etc.

### With Generics

```typescript
function getFirst<T>(arr: T[]): T {
  return arr[0];
}
```

Explanation:

- `T[]` means an array of type T.
- The function returns a value of type T.

Example:

```typescript
const n = getFirst([1, 2, 3]);          // number
const s = getFirst(["a", "b", "c"]);    // string
```

One function works for all types.

# 7. Generics with Interfaces

Generics are often used when defining reusable data structures.

Example: API responses.

Many APIs return a similar structure:

```
{
  data: something,
  success: true/false
}
```

But `data` can contain different types.

Instead of repeating interfaces, we use generics.

```typescript
interface ApiResponse<T> {
  data: T;
  success: boolean;
  message?: string;
}
```

Usage:

```typescript
interface User {
  id: number;
  name: string;
}

const response: ApiResponse<User> = {
  data: { id: 1, name: "Dania" },
  success: true
};
```

The structure is reusable for many types.

# 8. Generics with Type Aliases

Generics can also be used with type aliases.

Example:

```typescript
type Box<T> = {
  value: T;
};
```

Usage:

```typescript
const stringBox: Box<string> = {
  value: "Hello"
```

```
};

const numberBox: Box<number> = {
  value: 10
};
```

The same type definition works for multiple types.

# 9. Generics in Classes

Classes can also be generic.

Example:

```TypeScript
class Storage<T> {

  private items: T[] = [];

  add(item: T): void {
    this.items.push(item);
  }

  getAll(): T[] {
    return this.items;
  }
}
```

Usage:

```TypeScript
const numberStorage = new Storage<number>();
numberStorage.add(10);
numberStorage.add(20);

const stringStorage = new Storage<string>();
stringStorage.add("Hello");
```

The same class works with different types.

# Objects in TypeScript

# Objects in JavaScript

Before understanding objects in TypeScript, we need to understand how they work in JavaScript.

### Example

```javascript
const user = {
  name: "Dania",
  age: 28,
  isAdmin: true
};
```

This object contains three properties.

| Property | Value |
|---|---|
| name | "Dania" |
| age | 28 |
| isAdmin | true |

Each property follows this structure:

```
None
key : value
```

# The Problem in JavaScript

JavaScript does not enforce types.

For example:

```javascript
const user = {
  name: "Dania",
  age: 28
};

user.age = "twenty eight";
```

JavaScript allows this change.

This may cause **unexpected bugs** in large applications.

Another problem occurs when properties are missing.

```javascript
function printUser(user) {
  console.log(user.name);
}

printUser({});
```

This prints:

```
None
undefined
```

The program runs but behaves incorrectly.

This is where **TypeScript helps**.

# Objects in TypeScript

TypeScript allows us to define **the structure of an object**.

We can specify:

- which properties exist
- the type of each property

Example:

```typescript
const user: { name: string; age: number } = {
  name: "Dania",
  age: 28
};
```

Explanation:

```
name: string
age: number
```

This means:

- name must be a string
- age must be a number

If we try this:

```typescript
const user: { name: string; age: number } = {
  name: "Dania",
  age: "28"
};
```

TypeScript produces an error.

# Using Object Types in Functions

Objects are frequently passed to functions.

Example:

```
TypeScript
function printUser(user: { name: string; age: number }) {
  console.log(user.name);
}
```

Usage:

```
TypeScript
printUser({
  name: "Ali",
  age: 25
});
```

TypeScript ensures that the object passed to the function matches the required structure.

However, writing object types repeatedly becomes messy.

# 6. Using Interfaces (Best Practice)

Instead of rewriting object structures, we can define them once using an **interface**.

### Example

```
TypeScript
interface User {
  name: string;
  age: number;
}
```

Now we can reuse it.

```typescript
const user: User = {
  name: "Dania",
  age: 28
};
```

Or inside functions:

```typescript
function printUser(user: User) {
  console.log(user.name);
}
```

Interfaces improve:

- readability
- maintainability
- reusability

# Optional Properties

Sometimes an object property is **not required**.

Example:

```typescript
interface User {
  name: string;
  age: number;
  email?: string;
}
```

The `?` symbol means the property is optional.

Example usage:

```typescript
const user: User = {
  name: "Dania",
  age: 28
};
```

This is valid.

But if we include email:

```typescript
const user: User = {
  name: "Dania",
  age: 28,
  email: "dania@email.com"
};
```

It must still be a string.

# Readonly Properties

Some properties should not change after creation.

Example:

```typescript
interface User {
  readonly id: number;
  name: string;
}
```

Usage:

```typescript
const user: User = {
  id: 1,
  name: "Ali"
```

```
};
```

Trying to modify `id` will cause an error.

```TypeScript
user.id = 2; // Error
```

Readonly protects critical data.

# Nested Objects

Objects can contain other objects.

Example:

```TypeScript
interface Address {
  city: string;
  country: string;
}

interface User {
  name: string;
  address: Address;
}
```

Usage:

```TypeScript
const user: User = {
  name: "Dania",
  address: {
    city: "Jerusalem",
    country: "Palestine"
  }
};
```

Accessing nested values:

```
TypeScript
console.log(user.address.city);
```

# 10. Objects with Functions

Objects can contain functions.

These functions are called **methods**.

Example:

```
TypeScript
interface User {
  name: string;
  greet(): void;
}
```

Usage:

```
TypeScript
const user: User = {
  name: "Dania",

  greet() {
    console.log("Hello " + this.name);
  }
};
```

Calling the method:

```
TypeScript
user.greet();
```

# What is `Omit` in TypeScript?

`Omit` is a **utility type** provided by TypeScript.

It allows you to:

> Create a new type by removing one or more properties from another type.

In other words:

**Take a type → remove some keys → create a new type**

## Basic Syntax

```typescript
Omit<Type, Keys>
```

Where:

- **Type** → the original type or interface
- **Keys** → the properties you want to remove

## Simple Example

First we define an interface.

```typescript
interface User {
  id: number
  name: string
  email: string
  password: string
}
```

Now imagine we want to return user data from an API.

But we **must not expose the password**.

Instead of creating a new interface manually, we use `Omit`.

```
TypeScript
type PublicUser = Omit<User, "password">
```

Now the new type becomes:

```
TypeScript
{
  id: number
  name: string
  email: string
}
```

The `password` property is removed.

# Using the New Type

```
TypeScript
const user: PublicUser = {
  id: 1,
  name: "Dania",
  email: "dania@email.com"
}
```

If we try to add password:

```
TypeScript
const user: PublicUser = {
  id: 1,
  name: "Dania",
  email: "dania@email.com",
  password: "123"
}
```

TypeScript will show an error.

Because `password` is omitted.

# Omitting Multiple Properties

You can remove multiple fields.

Example:

```typescript
type UserPreview = Omit<User, "email" | "password">
```

Now the new type is:

```typescript
{
  id: number
  name: string
}
```

# Why `Omit` is Useful

`Omit` helps us avoid repeating code.

Without `Omit` we would write:

```typescript
interface PublicUser {
  id: number
  name: string
  email: string
}
```

But if the `User` interface changes later, we must update multiple places.

Using `Omit` ensures the types stay connected.

# Real World Example

Imagine a **database model**:

```typescript
interface User {
  id: number
  name: string
  email: string
  password: string
  createdAt: Date
}
```

When creating a new user, the server usually generates:

- `id`
- `createdAt`

So we omit them:

```typescript
type CreateUser = Omit<User, "id" | "createdAt">
```

Now the required data becomes:

```typescript
{
  name: string
  email: string
  password: string
}
```

# Pick<Type, Keys>

## The Idea

`Pick` allows us to **create a new type by selecting specific properties from an existing type**.

Instead of rewriting a new interface manually, we **pick only the fields we want**.

### Syntax

```typescript
Pick<Type, Keys>
```

Where:

- **Type** → the original type
- **Keys** → the properties we want to extract

## Example 1 — Basic Object

Let's start with a simple interface.

```typescript
interface User {
```

```
  id: number;

  name: string;

  email: string;

  password: string;

}
```

Now imagine we want to return **user data to the frontend**, but we **must never send the password**.

Instead of writing a new interface manually:

```TypeScript
interface PublicUser {

  id: number;

  name: string;

  email: string;

}
```

We can use **Pick**.

```TypeScript
type PublicUser = Pick<User, "id" | "name" | "email">;
```

## What happens here?

TypeScript creates a **new type** that includes **only the selected keys**.

Result:

```TypeScript
type PublicUser = {
```

```
  id: number;
  name: string;
  email: string;
}
```

## Code Example

```typescript
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

/*
Pick will create a new type
that includes only id and name
*/
type UserPreview = Pick<User, "id" | "name">;

const user: UserPreview = {
  id: 1,
  name: "Dania",
```

```
};
```

Notice:

- `email` is not allowed
- `password` is not allowed

TypeScript will show an error if we try to add them.

# Real World Use Cases

Pick is commonly used when:

## 1 API responses

You don't want to expose everything.

Example:

```TypeScript
type UserResponse = Pick<User, "id" | "name" | "email">;
```

## 2 Component props

Maybe a component only needs **name and email**.

```TypeScript
type UserCardProps = Pick<User, "name" | "email">;
```

## 3 Database queries

Sometimes queries return only specific columns.

# 2. Partial

## The Idea

`Partial` makes **all properties optional**.

This is extremely useful when performing **updates**.

## Syntax

```typescript
Partial<Type>
```

## Example

Original interface:

```typescript
interface User {
  id: number;
  name: string;
  email: string;
}
```

Normally TypeScript requires **all fields**.

```typescript
const user: User = {
  id: 1,
```

```
  name: "Dania",
  email: "dania@mail.com"
};
```

But when updating a user, maybe we want to update **only the name**.

This would normally fail.

Instead we use **Partial**.

## Example Code

```typescript
interface User {
  id: number;
  name: string;
  email: string;
}

/*
Partial makes all properties optional
*/
type UpdateUser = Partial<User>;

const update: UpdateUser = {
  name: "New Name"
};
```

Now TypeScript allows this.

## What TypeScript Generates

```typescript
type UpdateUser = {
  id?: number;
  name?: string;
  email?: string;
}
```

Notice the **?**

Everything becomes **optional**.

## Real World Example — API Update

```typescript
function updateUser(userId: number, updates: Partial<User>) {
  // updates may contain only some fields
}
```

Usage:

```typescript
updateUser(1, { name: "Ali" });
updateUser(1, { email: "ali@mail.com" });
```

```
updateUser(1, { name: "Ali", email: "ali@mail.com" });
```

# 3. Required

## The Idea

`Required` does the **opposite of Partial**.

It makes **all optional properties required**.

## Syntax

```TypeScript
Required<Type>
```

## Example

Original type:

```TypeScript
interface User {
  id: number;
  name?: string;
  email?: string;
}
```

Here:

- `name` optional
- `email` optional

But maybe in a certain context we **require all fields**.

## Example Code

```typescript
interface User {
  id: number;
  name?: string;
  email?: string;
}

/*
Required converts optional properties
to required properties
*/
type CompleteUser = Required<User>;

const user: CompleteUser = {
  id: 1,
  name: "Dania",
```

```
  email: "dania@mail.com"
};
```

Now if we omit `name` or `email`, TypeScript throws an error.

## What TypeScript Generates

```TypeScript
type CompleteUser = {
  id: number;
  name: string;
  email: string;
}
```

Everything becomes **required**.

# Visual Understanding

### Original Type

```TypeScript
interface User {
  id: number;
  name: string;
  email: string;
```

```
}
```

### Pick

```typescript
type A = Pick<User, "id" | "name">;
```

Result

```typescript
{
  id: number
  name: string
}
```

### Partial

```typescript
type B = Partial<User>;
```

Result

```typescript
{
  id?: number
  name?: string
  email?: string
}
```

### Required

```typescript
type C = Required<User>;
```

Result

```typescript
{
  id: number
  name: string
  email: string
}
```

```typescript
```

# TypeScript with Promises and React

## 1. Why this topic matters

**So far, TypeScript may look like:**

- **typing variables**
- **typing functions**
- **typing objects**

**But in real projects, we often work with:**

- **asynchronous operations**
- **API calls**
- **components**
- **props**
- **state**
- **events**

**That means TypeScript becomes especially valuable in:**

- **Promises**
- **React**

# Part 1 — TypeScript with Promises

## 2. What is a Promise?

**A Promise is an object that represents the result of an asynchronous operation.**

**It usually has 3 states:**

- **pending**
- **fulfilled**
- **rejected**

**Example in JavaScript:**

```javascript
const promise = new Promise((resolve, reject) => {
  resolve("Done");
});
```

**When we use TypeScript, the important question becomes:**

> **What type of value will this Promise eventually return?**

**That is where TypeScript helps.**

## 3. Promise with a type

**In TypeScript, a Promise is usually written like this:**

```typescript
Promise<string>
```

**This means:**

> **This Promise will eventually resolve to a string.**

**Example:**

```typescript
const messagePromise: Promise<string> = new
Promise((resolve, reject) => {
  resolve("Hello from TypeScript");
});
```

**Explanation:**

- `Promise<string>` **means the resolved value must be a string**
- **If we try to resolve a number, TypeScript will complain**

**Example:**

```typescript
const messagePromise: Promise<string> = new
Promise((resolve) => {
  resolve(100); // Error
});
```

**Because the Promise promised to return a string, not a number.**

## 4. Promise with number

```typescript
const numberPromise: Promise<number> = new
Promise((resolve) => {
  resolve(42);
});
```

**Here the resolved value is a number.**

## 5. Promise with objects

**This is where it becomes very useful.**

**Example:**

```typescript
interface User {
  id: number;
  name: string;
}

const userPromise: Promise<User> = new
Promise((resolve) => {
  resolve({
    id: 1,
```

```
    name: "Dania",
  });
});
```

Now TypeScript knows that the Promise resolves to a `User` object.

## 6. Using `.then()` with TypeScript

```TypeScript
userPromise.then((user) => {
  console.log(user.name);
});
```

TypeScript already knows:

- `user` is of type `User`

So it gives:

- autocomplete
- property checking
- error detection

If we write:

```TypeScript
userPromise.then((user) => {
  console.log(user.email);
});
```

TypeScript gives an error if `email` is not part of `User`.

That is a huge benefit.

## 7. Async / await with TypeScript

**This is the most common real-world pattern.**

**Example:**

```typescript
interface Product {
  id: number;
  title: string;
}

async function getProduct(): Promise<Product> {
  return {
    id: 1,
    title: "Laptop",
  };
}
```

**Explanation:**

- `async` functions always return a Promise
- `Promise<Product>` means the function resolves to a `Product`

**Now usage:**

```typescript
async function printProduct() {
  const product = await getProduct();
  console.log(product.title);
}
```

**TypeScript understands that `product` is `Product`.**

## 8. Why Promise typing is important

**Without TypeScript, when you fetch data or use promises:**

- **you may not know the shape of the result**
- **you may use wrong properties**
- **you may assume wrong types**

**With TypeScript:**

- **the returned data shape is clear**
- **misuse is caught early**
- **autocomplete becomes smarter**

## 9. Real API-style example

```typescript
interface Todo {
  id: number;
  title: string;
  completed: boolean;
}

async function fetchTodo(): Promise<Todo> {
  return {
    id: 1,
    title: "Study TypeScript",
    completed: false,
  };
}
```

**Usage:**

```typescript
async function showTodo() {
  const todo = await fetchTodo();
  console.log(todo.title);
```

```
}
```

This is exactly the kind of pattern students will see in real projects.

# Part 2 — TypeScript with React

## 10. Why TypeScript is powerful in React

React applications work with many structured things:

- components
- props
- state
- events
- API responses
- refs

Without TypeScript, mistakes are easy:

- wrong prop names
- wrong prop types
- wrong event handling
- wrong state shape

TypeScript gives React components contracts and clarity.

## 11. Typing props in React

This is usually the first important topic.

Imagine a component:

```
None
function UserCard(props) {
  return <h1>{props.name}</h1>;
}
```

**In plain JavaScript, nothing tells us what `props` should contain.**

**In TypeScript, we define it.**

**Using a type alias**

```
None
type UserCardProps = {
  name: string;
  age: number;
};
```

**Then:**

```
None
function UserCard(props: UserCardProps) {
  return (
    <div>
      <h1>{props.name}</h1>
      <p>{props.age}</p>
    </div>
  );
}
```

**Usage:**

```
None
<UserCard name="Dania" age={28} />
```

**If we write:**

```
None
<UserCard name="Dania" age="28" />
```

**TypeScript gives an error, because `age` must be a number.**

## 12. Destructuring props with TypeScript

**This is cleaner and more common.**

```
None
type UserCardProps = {
  name: string;
  age: number;
};

function UserCard({ name, age }: UserCardProps) {
  return (
    <div>
      <h1>{name}</h1>
      <p>{age}</p>
    </div>
  );
}
```

## 13. Optional props in React

**Sometimes a prop is not required.**

```
None
type ButtonProps = {
  label: string;
  color?: string;
};
```

**Component:**

```
None
function Button({ label, color }: ButtonProps) {
  return <button>{label} - {color}</button>;
}
```

**Usage:**

```
None
<Button label="Save" />
<Button label="Delete" color="red" />
```

**Because `color` is optional.**

## 14. Typing state in React

**useState with TypeScript**

**Example:**

```
None
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState<number>(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

**Here:**

- `count` **is number**
- `setCount` **expects numbers**

**If we try:**

```
None
setCount("hello");
```

**TypeScript gives an error.**

## 15. useState with objects

```
None
type User = {
  name: string;
  age: number;
};

function Profile() {
  const [user, setUser] = useState<User>({
    name: "Dania",
    age: 28,
  });

  return <h1>{user.name}</h1>;
}
```

**This makes state safer and clearer.**

## 16. useState with arrays

```
None
function Students() {
  const [names, setNames] =
useState<string[]>(["Ali", "Sara"]);

  return (
    <ul>
      {names.map((name) => (
        <li key={name}>{name}</li>
```

```
      ))}
    </ul>
  );
}
```

**Now `names` is typed as an array of strings.**

```
None
```

## 17. Typing async data in React

**Very common real-world pattern:**

```
None
import { useEffect, useState } from "react";

type Todo = {
  id: number;
  title: string;
  completed: boolean;
};

function TodoList() {
  const [todos, setTodos] = useState<Todo[]>([]);

  useEffect(() => {
    async function loadTodos() {
      const data: Todo[] = [
        { id: 1, title: "Study", completed: false },
```

```
        { id: 2, title: "Practice", completed: true
},
      ];

      setTodos(data);
    }

    loadTodos();
  }, []);

  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.title}</li>
      ))}
    </ul>
  );
}
```

**This is a perfect example of TypeScript + Promise + React together.**

## 19. Promise + React together

**This is where everything connects.**

**Example**

```
None
type Product = {
  id: number;
  title: string;
};
```

```
async function fetchProducts(): Promise<Product[]> {
  return [
    { id: 1, title: "Laptop" },
    { id: 2, title: "Phone" },
  ];
}
```

**Inside a component:**

```None
import { useEffect, useState } from "react";

function ProductsList() {
  const [products, setProducts] =
useState<Product[]>([]);

  useEffect(() => {
    async function loadProducts() {
      const data = await fetchProducts();
      setProducts(data);
    }

    loadProducts();
  }, []);

  return (
    <ul>
      {products.map((product) => (
        <li key={product.id}>{product.title}</li>
      ))}
    </ul>
  );
}
```

**Now TypeScript knows:**

- `fetchProducts()` **returns** `Promise<Product[]>`
- `data` **becomes** `Product[]`
- `products` **state is also** `Product[]`

**Everything stays consistent.**